



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

FAKULTA RIADENIA A INFORMATIKY UNIZA

Semestrálna práca - Idle Fishing

Fakulta riadenia a informatiky, UNIZA

Predmet: Informatika 2

Rebeca Martinaková

`martinakova1@stud.uniza.sk`

17.5.2026

1 Prehľad projektu

Idle Fishing je idle-clicker hra napísaná v jazyku Java s grafickým rozhraním postaveným na knižnici Swing inšpirovaná populárnou hrou Cookie Clicker. Hráč chytá ryby kliknutím na prút, zarába pasívny príjem pomocou plavidiel, odomýká nové lokality a nakupuje vylepšenia. Po dosiahnutí určitej sumy je možné spustiť prestíž, ktorá navždy zvyšuje celkový príjem.

Hlavná herná slučka

1. Hráč klikne na prút → chytí rybu → získa peniaze.
2. Hráč kúpi plavidlá → automatický príjem každú sekundu.
3. Hráč odomkne nové miesto → vyšší základ kliknutia, vzácnejšie ryby.
4. Hráč kúpi vylepšenia → násobí výstup plavidiel alebo hodnotu kliknutia.
5. Pri celkovom zarábku €1 000 000 je dostupná prestíž (+10% trvalý bonus).

2 Zostavenie a spustenie

2.1 Požiadavky

- Java 11 JDK (OpenJDK alebo Oracle)
- Java 21 LTS
- Apache Maven 3.6+

2.2 Zostavenie

```
mvn package
```

Výsledok: `target/IdleFishing.jar`

2.3 Spustenie

```
java -jar target/IdleFishing.jar
```

Ukladací súbor `idlefishing.sav` sa zapisuje do pracovného adresára. Zmazaním tohto súboru sa hra resetuje.

3 Herná mechanika

3.1 Aktívny rybolov

Kliknutie na prút spustí `FishCommand`. Príkaz:

1. Vypočíta hodnotu kliknutia aktuálnej lokality ($\text{základ} \times \text{multiplikátor} \times \text{level bonus}$).
2. Aplikuje náhodnú odchýlku $\pm 25\%$.
3. Aplikuje prestížny bonus.
4. Vyberie náhodnú rybu z váhového bazéna lokality.
5. Pridá vypočítanú hodnotu k peniazom hráča.

3.2 Pasívny príjem

Každú sekundu sa spustí časovač v `MainFrame`. Každé plavidlo zavolá `produce()`, ktoré deleguje výpočet na svoju stratégiu:

Plavidlo	Stratégia	Vzorec
Rowboat, FishingBoat	Lineárna	$\text{baseRate} \times \text{count}$
Trawler	Logaritmická	$\text{baseRate} \times \text{count} \times \log_2(\text{count} + 1)$
FactoryShip	Exponenciálna	$\text{baseRate} \times \text{count} \times 1.05^{\text{count}-1}$

Tabuľka 1: Produkčné stratégie plavidiel

Celkový príjem za sekundu sa ešte násobí prestížnym bonusom.

Lokalita	Cena	Základ kliknutia	Ryby
River	zadarmo	€2	len bežné ryby
Lake	€500	€9	bežné + vzácne
Sea	€10 000	€35	+ legendárne
Ocean	€100 000	€120	+ exotické

Tabuľka 2: Herné lokality

3.3 Lokality

3.4 Vzácnosť rýb

- **Common** — sivá, nízka hodnota, časté
- **Rare** — modrá, stredná hodnota
- **Legendary** — zlatá, vysoká hodnota, zriedkavé
- **Exotic** — ružová, veľmi vysoká hodnota, veľmi zriedkavé

Hodnota ryby sa navyše zvyšuje o 10 % za každý kilogram hmotnosti nad 1 kg.

3.5 Rast ceny plavidiel

Každý nákup zdrazí ďalšiu jednotku:

$$currentCost = baseCost \times 1,15^{count}$$

3.6 Vylepšenia

K dispozícii je 20 vylepšení v dvoch kategóriách:

- **VesselUpgrade** — násobí produkciu konkrétneho plavidla.
- **SpotUpgrade** — násobí hodnotu kliknutia konkrétnej lokality.

Každé vylepšenie sa dá kúpiť iba raz.

3.7 Prestíž

Prestíž sa odomkne, keď `totalEarned` dosiahne €1 000 000. Po stlačení:

1. Zvýši sa `prestigeLevel`.
2. Peniaze, plavidlá, vylepšenia a lokalita sa resetujú.
3. Trvalo sa aplikuje: $prestigeBonus = 1,1^{prestigeLevel}$
4. Všetok budúci príjem (klik aj pasívny) sa násobí týmto bonusom.

3.8 Offline zárobky

Pri načítaní hry sa vypočíta uplynulý čas od posledného uloženia:

$$offlineEarnings = \min(elapsed, 8h) \times productionPerSecond$$

Dialóg informuje hráča o výške offline zárobku.

4 Architektúra

Projekt využíva vrstvenú štruktúru podobnú MVC:

`model/` Čistý herný stav, bez závislosti na UI alebo I/O.

`strategy/` Matematické algoritmy produkcie.

`observer/` Event bus; oddeľuje model od UI.

`command/` Akcie hráča ako objekty; píšú do `GameState`, publikujú udalosti.

`factory/` Vytváranie objektov; `GameState` nevolá `new` priamo.

`persistence/` Ukladanie a načítanie.

`ui/` Všetky Swing panely; odoberajú udalosti, nevolajú model priamo.

`util/` Zdieľané konštanty a farebná paleta.

Tok dát pri kliknutí hráča

FishingPanel (klik)

```
-> FishCommand.execute()
-> GameState.addMoney()
-> EventBus.publish(FISH_CAUGHT)
-> FishingPanel, StatsPanel (aktualizcia zobrazenia)
```

Tok dát pri pasívnom príjme

MainFrame Timer (1 s)

```
-> GameState.getTotalProductionPerSecond()
-> GameState.addMoney()
-> EventBus.publish(MONEY_CHANGED)
-> UI panely sa obnovia
```

5 Návrhové vzory

5.1 Singleton

- **GameState** — jediný zdroj pravdy pre všetky herné dáta.
- **EventBus** — jeden zdieľaný pub-sub hub pre celú aplikáciu.
- **SaveManager** — jediný vstupný bod pre všetky operácie ukladania.

5.2 Factory Method

SpotFactory, VesselFactory a UpgradeFactory poskytujú statickú metódu `createAll()`. `GameState` volá tieto továrne namiesto priameho použitia `new`, čím je logika konštrukcie izolovaná.

5.3 Strategy

Rozhranie `ProductionStrategy` má tri implementácie:

- **Linear** — pre skoré plavidlá, predvídateľný rast.
- **Logarithmic** — stredná hra; klesajúce výnosy na jednotku.
- **Exponential** — neskorá hra; rastúci výnos na jednotku.

`Vessel` drží referenciu; `produce()` deleguje bez znalosti vzorca.

5.4 Template Method

`FishingSpot.catchFish()` definuje kostru algoritmu. Abstraktná metóda `buildFishPool()` je implementovaná každou konkrétnou podtriedou, čím riadi, aké druhy rýb sa na danej lokalite objavujú.

Podobne `Vessel.produce()` volá abstraktné `getBaseProductionRate()` a skalárnu časť deleguje na injektovanú stratégiu.

5.5 Observer / Pub-Sub

`EventBus` mapuje `EventType` na zoznam `GameObserver`. UI panely sa prihlásia na udalosti, ktoré ich zaujímajú. Command objekty publikujú udalosti po zmene stavu. Vďaka tomu model neimportuje žiadnu Swing triedu.

5.6 Command

Každá akcia hráča (rybolov, nákup plavidla, nákup vylepšenia, odomknutie lokality, prestíž) je zapuzdrená v objekte implementujúcom rozhranie `Command` s metódami `execute() : boolean` a `getDescription()`. `CommandHistory` uchováva posledných 100 vykonaných príkazov.

6 Prehľad tried

6.1 com.idlefishing.model.fish

Fish (abstraktná) Základná trieda. Polia: `name`, `baseValue`, `weight`. Metóda `calculateValue()` aplikuje bonus za hmotnosť.

CommonFish Vzácnosť *Common*, sivá farba.

RareFish Vzácnosť *Rare*, modrá farba.

LegendaryFish Vzácnosť *Legendary*, zlatá farba.

ExoticFish Vzácnosť *Exotic*, ružová farba.

6.2 com.idlefishing.model.spot

FishingSpot (abstraktná) Template Method: `catchFish()` / `buildFishPool()`.

RiverSpot Zadarmo, základ kliknutia 2,0.

LakeSpot €500, základ kliknutia 9,0.

SeaSpot €10 000, základ kliknutia 35,0.

OceanSpot €100 000, základ kliknutia 120,0.

6.3 com.idlefishing.model.vessel

Vessel (abstraktná) Drží `count`, stratégiu a `productionMultiplier`.

Rowboat Základná sadzba 1,0/s, lineárna stratégia.

FishingBoat Základná sadzba 12,0/s, lineárna stratégia.

Trawler Základná sadzba 130,0/s, logaritmická stratégia.

FactoryShip Základná sadzba 1500,0/s, exponenciálna stratégia.

6.4 com.idlefishing.model.upgrade

Upgrade (abstraktná) Polia: id, name, cost, purchased.

VesselUpgrade Násobí produkciu konkrétneho plavidla.

SpotUpgrade Násobí hodnotu kliknutia konkrétnej lokality.

6.5 com.idlefishing.model

GameState (Singleton) Centrálny stav: peniaze, celkové zárobky, počet rýb, úroveň prestíže, bonusy, zoznamy lokalít, plavidiel a vylepšení.

6.6 com.idlefishing.strategy

ProductionStrategy Funkčné rozhranie: `calculate(baseRate, count)`.

LinearProductionStrategy $baseRate \times count$

LogarithmicProductionStrategy $baseRate \times count \times \log_2(count + 1)$

ExponentialProductionStrategy $baseRate \times count \times 1,05^{count-1}$

6.7 com.idlefishing.observer

GameObserver Rozhranie: `onEvent(GameEvent)`.

EventBus (Singleton) `subscribe()` a `publish()`.

GameEvent Obaľuje typ, dátové zaťaženie a časovú pečiatku.

EventType (enum) FISH_CAUGHT, MONEY_CHANGED, VESSEL_PURCHASED, UPGRADE_PURCHASED, LOCATION_CHANGED, LOCATION_UNLOCKED, PRESTIGE_TRIGGERED, GAME_TICK, GAME_SAVED, GAME_LOADED, OFFLINE_EARNINGS.

6.8 com.idlefishing.command

Command Rozhranie: `execute() : boolean, getDescription()`.

FishCommand Chytí rybu, pridá peniaze, publikuje udalosť.

BuyVesselCommand Odpočíta cenu, zavolá `vessel.purchase()`.

BuyUpgradeCommand Odpočíta cenu, aplikuje multiplikátor na cieľ.

UnlockLocationCommand Odpočíta cenu, odomkne lokalitu.

PrestigeCommand Zavolá `GameState.prestige()`.

CommandHistory Deque posledných 100 príkazov.

6.9 com.idlefishing.factory

SpotFactory `createAll()` → 4 inštancie `FishingSpot`.

VesselFactory `createAll()` → 4 inštancie `Vessel`.

UpgradeFactory `createAll()` → 20 inšancií `Upgrade`.

6.10 com.idlefishing.persistence

SaveManager (Singleton) Ukladá a načítava stav hry ako `.properties` súbor. Pri načítaní vypočíta offline zárobky.

6.11 com.idlefishing.ui

MainFrame Hlavné okno. Drží časovač herného tikú (1 s) a časovač automatického ukládania (60 s).

FishingPanel Ľavý panel. Animovaná scéna s loďami a klikateľným prútom.

ShopPanel Pravý panel (340 px). Riadky plavidiel s ikonami, scrollovateľný zoznam vylepšení.

LocationBar Horná lišta so 4 tlačidlami lokalít.

StatsPanel Spodná lišta: €/s, počet rýb, úroveň prestíže, tlačidlo prestíže.

VesselPainter Statická utilita. Kreslí ľubovoľné plavidlo cez Java2D na zadanej pozícii, škále a výškovom ofsete.

`VesselIconPanel` Panel 92×70 px používaný v `ShopPanel` pre náhľady plavidiel.

6.12 `com.idlefishing.util`

`GameConstants` Všetky číselné vyvážovacie hodnoty.

`UIColors` Tmavá farebná paleta a fontové konštanty.

7 Systém ukladania

Súbor: `idlefishing.sav` (formát Java `.properties`, čistý text)

Ukladané polia

- `money`, `totalEarned`, `totalFishCaught`
- `prestigeLevel`, `prestigeBonus`
- `currentSpotIndex`
- Pre každú lokalitu: `unlocked`, `level`, `clickMultiplier`
- Pre každé plavidlo: `count`, `productionMultiplier`
- Pre každé vylepšenie: príznak `purchased`
- `saveTimestamp` (epoch milisekundy)

Postup pri načítaní

1. Obnovia sa všetky polia.
2. $elapsed = currentTimeMillis - saveTimestamp$
3. $offlineSeconds = \min(elapsed/1000, 8 \times 3600)$
4. $offlineEarnings = offlineSeconds \times productionPerSecond$
5. Zárobky sa pripočítajú k peniazom.
6. Dialóg informuje hráča o výške offline zárobku.

Ak súbor chýba alebo je poškodený, hra začne odznova.

8 Vyvážovacie konštanty

Konštantá	Hodnota	Popis
PRICE_MULTIPLIER	1,15	Škálovanie ceny pri každom nákupe
MAX_OFFLINE_HOURS	8	Maximálna doba offline zárobkov
PRESTIGE_THRESHOLD	1 000 000	Celkový zárobok pre prestíž
CLICK_VARIANCE	0,50	Rozsah náhodnej odchýlky $\pm 25\%$
LEVEL_CLICK_BONUS	0,15	+15 % hodnoty kliknutia za level lokality
WEIGHT_VALUE_BONUS	0,10	+10 % hodnoty ryby za kg nad 1 kg
<i>Plavidlá</i>		
Rowboat	baseCost 15, baseRate 1,0/s	
FishingBoat	baseCost 100, baseRate 12,0/s	
Trawler	baseCost 1100, baseRate 130,0/s	
FactoryShip	baseCost 12000, baseRate 1500,0/s	

Tabuľka 3: Herné konštanty

9 Polymorfizmus

Polymorfizmus (z gréc. *polys* = veľa, *morphē* = tvar) je jeden zo základných pilierov objektovo orientovaného programovania (OOP) spolu s dedičnosťou a zapuzdrením. Umožňuje, aby rôzne objekty rôznych tried reagovali na rovnakú správu (volanie metódy) každý po svojom. Vďaka polymorfizmu môže kód pracovať s abstrakciami (rozhraniami, abstraktnými triedami) a zostáva funkčný bez ohľadu na to, aký konkrétny typ sa v kolekcii alebo premennej nachádza [1].

V jazyku Java rozlišujeme dva hlavné druhy polymorfizmu:

Compile-time (statický) polymorfizmus Realizovaný preťažením metód (*method overloading*). Kompilátor vyberie správnu verziu metódy na základe počtu a typov parametrov ešte pred spustením programu [2].

Runtime (dynamický) polymorfizmus Realizovaný prepísaním metód (*method overriding*) a mechanizmom *virtual dispatch*. JVM rozhodne za behu, ktorú konkrétnu implementáciu má zavolať, podľa skutočného (dynamického) typu objektu, nie podľa deklarovaného (statického) typu referencie. Práve tento druh polymorfizmu je v projekte Idle Fishing dominantný a je podrobne popísaný v nasledujúcich podkapitolách [3].

Projekt Idle Fishing obsahuje **7 polymorfických hierarchií**: 3 abstraktné triedy a 4 rozhrania s celkovo 22 konkrétnymi implementáciami.

9.1 Hierarchia rýb — abstraktná trieda Fish

Súbor: src/main/java/com/idlefishing/model/fish/Fish.java

Abstraktná trieda Fish definuje spoločné rozhranie pre všetky druhy rýb v hre. Obsahuje tri abstraktné metódy, ktoré musí každá podtrieda povinne implementovať:

Listing 1: Abstraktná trieda Fish

```
public abstract class Fish {
    protected String name;
    protected double baseValue;
    protected double weight;

    public abstract String getRarity();
    public abstract Color getDisplayColor();
    public abstract String getCatchMessage();

    public double calculateValue(double clickValue, double prestigeBonus) {
        double weightBonus = Math.max(0, (weight - 1.0) * GameConstants.
WEIGHT_VALUE_BONUS);
        return (baseValue + clickValue * 0.1) * (1 + weightBonus) *
prestigeBonus;
    }
}
```

Podtriedy a ich polymorfické implementácie:

Trieda	getRarity()	getDisplayColor()	getCatchMessage()
CommonFish	<i>Common</i>	sivá	obyčajná správa
RareFish	<i>Rare</i>	modrá	správa o vzácnom úlovku
LegendaryFish	<i>Legendary</i>	zlatá	správa o legendárnom úlovku
ExoticFish	<i>Exotic</i>	ružová	správa o exotickom úlovku

Tabuľka 4: Polymorfické implementácie podtried Fish

Polymorfizmus za behu: Každá lokalita (`FishingSpot`) udržiava zoznam `List<Fish>` `fishPool`, ktorý obsahuje mix rôznych konkrétnych podtried. Keď hráč klikne, zavolá sa:

Listing 2: Polymorfické volanie metód na kolekcii `Fish`

```
Fish caught = fishPool.get(random.nextInt(fishPool.size()));
String rarity = caught.getRarity();           // dispatch na konkrétny typ
Color color = caught.getDisplayColor();       // dispatch na konkrétny typ
String message = caught.getCatchMessage();    // dispatch na konkrétny typ
```

JVM automaticky smeruje každé volanie na správnu podtriedu bez toho, aby volajúci kód vedel, s akým konkrétnym typom pracuje. Toto je klasický prípad *runtime polymorfizmu* prostredníctvom *virtual dispatch* [4].

9.2 Hierarchia plavidiel — abstraktná trieda `Vessel`

Súbor: `src/main/java/com/idlefishing/model/vessel/Vessel.java`

Abstraktná trieda `Vessel` definuje spoločné správanie pre všetky typy plavidiel. Okrem dvoch abstraktných metód obsahuje aj konkrétnu logiku nákupu, škálovania ceny a delegovania výpočtu produkcie na injektovanú stratégiu.

Listing 3: Abstraktná trieda `Vessel` — kľúčové metódy

```
public abstract class Vessel {
    public abstract double getBaseProductionRate();
    public abstract String getIcon();

    public double produce() {
        if (count == 0) return 0;
        return strategy.calculate(getBaseProductionRate(), count)
            * productionMultiplier;
    }

    public double getCurrentCost() {
        return baseCost * Math.pow(GameConstants.PRICE_MULTIPLIER, count);
    }
}
```

Štyri konkrétne podtriedy a ich polymorfické správanie:

Trieda	getBaseProductionRate()	getIcon()	Stratégia
Rowboat	1,0/s	<i>Rowboat</i>	Lineárna
FishingBoat	12,0/s	<i>Fishing Boat</i>	Lineárna
Trawler	130,0/s	<i>Trawler</i>	Logaritmická
FactoryShip	1500,0/s	<i>Factory Ship</i>	Exponenciálna

Tabuľka 5: Polymorfické implementácie podtried **Vessel**

Polymorfizmus za behu: Všetky štyri typy plavidiel sú uložené v jednom zozname `List<Vessel>` v `GameState`. Celkový pasívny príjem sa vypočíta jediným riadkom:

Listing 4: Polymorfická agregácia produkcie plavidiel

```
// GameState.java
double total = vessels.stream()
    .mapToDouble(Vessel::produce)
    .sum();
```

Prúd (stream) iteruje cez `Rowboat`, `FishingBoat`, `Trawler` aj `FactoryShip` rovnakým kódom — každý objekt vypočíta svoju produkciu podľa svojej implementácie `getBaseProductionRate()` a svojej injektovanej stratégie.

9.3 Hierarchia lokalít — abstraktná trieda `FishingSpot`

Súbor: `src/main/java/com/idlefishing/model/spot/FishingSpot.java`

Abstraktná trieda `FishingSpot` využíva vzor *Template Method* (pozri sekciu *Návrhové vzory*): metóda `catchFish()` definuje kostru algoritmu rybolovu, pričom abstraktná metóda `buildFishPool()` je ponechaná na implementáciu podtriedami. Tým každá lokalita riadi, aké druhy rýb sa na nej vyskytujú.

Listing 5: Template Method v `FishingSpot`

```
public abstract class FishingSpot {
    protected List<Fish> fishPool;

    protected abstract List<Fish> buildFishPool();
    public abstract double getBaseClickValue();
    public abstract String getEnvironmentFlavour();
```



```

public Fish catchFish() {
    if (fishPool == null) fishPool = buildFishPool(); // polymorficke volanie
    return fishPool.get(random.nextInt(fishPool.size()));
}

public double getClickValue() {
    return getBaseClickValue() // polymorficke volanie
        * clickMultiplier
        * (1 + level * GameConstants.LEVEL_CLICK_BONUS);
}
}

```

Štyri konkrétne lokality a ich polymorfická špecializácia:

Trieda	getBaseClickValue()	Ryby v bazéne
RiverSpot	2,0	Common, Rare, Legendary
LakeSpot	9,0	Common, Rare, Legendary
SeaSpot	35,0	Common, Rare, Legendary
OceanSpot	120,0	Rare, Legendary, Exotic

Tabuľka 6: Polymorfické implementácie podtried FishingSpot

Polymorfizmus za behu: BuyUpgradeCommand iteruje cez List<FishingSpot> a volá getClickValue() na každom prvku — kód nemusí poznať konkrétny typ:

Listing 6: Polymorfická iterácia cez lokality

```

for (FishingSpot spot : state.getFishingSpots()) {
    if (spot.getName().equals(upgrade.getTargetSpotName())) {
        spot.applyClickMultiplier(upgrade.getClickMultiplier());
    }
}

```

9.4 Hierarchia vylepšení — abstraktná trieda Upgrade

Súbor: src/main/java/com/idlefishing/model/upgrade/Upgrade.java

Abstraktná trieda `Upgrade` zapuzdruje spoločné vlastnosti všetkých vylepšení (id, meno, cena, stav kúpy). Abstraktné metódy `getCategory()` a `getEffectDescription()` si každá podtrieda implementuje sama.

Listing 7: Abstraktná trieda `Upgrade`

```
public abstract class Upgrade {
    public abstract String getCategory();
    public abstract String getEffectDescription();

    public boolean purchase() {
        if (purchased) return false;
        purchased = true;
        return true;
    }
}
```

`SpotUpgrade` Implementuje `getCategory()` → *"Spot"*. Obsahuje polia `targetSpotName` a `clickMultiplier`. V hre existuje 8 inštancií (2 pre každú lokalitu).

`VesselUpgrade` Implementuje `getCategory()` → *"Vessel"*. Obsahuje polia `targetVesselName` a `productionMultiplier`. V hre existuje 12 inštancií (3 pre každé plavidlo).

Polymorfizmus za behu: `ShopPanel` zobrazuje všetky vylepšenia z `List<Upgrade>` bez znalosti konkrétneho typu. Až `BuyUpgradeCommand` rozlíši typ cez `instanceof` a aplikuje príslušný efekt:

Listing 8: `instanceof` check pri aplikácii efektu vylepšenia

```
if (upgrade instanceof VesselUpgrade vu) {
    // najdi spravne plavidlo a aplikuj multiplikator
    for (Vessel v : state.getVessels()) {
        if (v.getName().equals(vu.getTargetVesselName()))
            v.applyProductionMultiplier(vu.getProductionMultiplier());
    }
} else if (upgrade instanceof SpotUpgrade su) {
    // najdi spravnu lokalitu a aplikuj multiplikator
    for (FishingSpot s : state.getFishingSpots()) {
```

```

        if (s.getName().equals(su.getTargetSpotName()))
            s.applyClickMultiplier(su.getClickMultiplier());
    }
}

```

Toto je príklad *downcasting* polymorfizmu — pracujeme s nadtriedou, ale v konkrétnom prípade potrebujeme prístup k špecifickým poliam podtriedy.

9.5 Rozhranie Command — vzor Command

Súbor: src/main/java/com/idlefishing/command/Command.java

Rozhranie Command je kľúčovým bodom polymorfizmu pre herné akcie. Každá akcia hráča — rybolov, nákup plavidla, nákup vylepšenia, odomknutie lokality, prestíž — je zapuzdrená v samostatnom objekte implementujúcom toto rozhranie:

Listing 9: Rozhranie Command

```

public interface Command {
    boolean execute();
    String getDescription();
}

```

Päť implementácií a ich polymorfické správanie:

Trieda	execute()	getDescription()
FishCommand	chytí rybu, pridá peniaze	<i>Fish at [lokalita]</i>
BuyVesselCommand	odpočíta cenu, kúpi plavidlo	<i>Buy [plavidlo]</i>
BuyUpgradeCommand	kúpi vylepšenie, aplikuje efekt	<i>Buy upgrade: [meno]</i>
UnlockLocationCommand	odomkne alebo prepne lokalitu	<i>Unlock/Switch [lokalita]</i>
PrestigeCommand	spustí prestíž	<i>Prestige → level N</i>

Tabuľka 7: Polymorfické implementácie rozhrania Command

Polymorfizmus za behu: CommandHistory uchováva históriu ako Deque<Command>. Aplikácia volá `command.execute()` a `command.getDescription()` na všetkých typoch rovnako:

Listing 10: Polymorfické volanie v CommandHistory

```
// UI vola kazdy prikaz rovnako:
boolean success = command.execute();           // runtime dispatch
String desc     = command.getDescription();    // runtime dispatch
history.addLast(command);                      // heterogenna kolekcia
```

Výhoda: pridanie novej akcie nevyžaduje úpravy existujúceho kódu — stačí vytvoriť novú triedu implementujúcu **Command**. Toto je *Open/Closed Principle* z SOLID zásad umožnený polymorfizmom [5].

9.6 Rozhranie ProductionStrategy — vzor Strategy

Súbor: src/main/java/com/idlefishing/strategy/ProductionStrategy.java

Funkčné rozhranie **ProductionStrategy** umožňuje za behu zameniť matematický vzorec výpočtu produkcie plavidla bez zmeny triedy **Vessel**:

Listing 11: Funkčné rozhranie ProductionStrategy

```
@FunctionalInterface
public interface ProductionStrategy {
    double calculate(double baseRate, int count);
}
```

Tri implementácie s odlišnou matematikou:

Trieda	Plavidlo	Vzorec
LinearProductionStrategy	Rowboat, FishingBoat	$baseRate \times count$
LogarithmicProductionStrategy	Trawler	$baseRate \times count \times \log_2(count + 1)$
ExponentialProductionStrategy	FactoryShip	$baseRate \times count \times 1,05^{count-1}$

Tabuľka 8: Polymorfické implementácie rozhrania **ProductionStrategy**

Polymorfizmus za behu: `Vessel` drží pole `ProductionStrategy` `strategy`. Metóda `produce()` deleguje výpočet bez znalosti konkrétnej implementácie:

Listing 12: Polymorfické delegovanie v `Vessel.produce()`

```
public double produce() {
    if (count == 0) return 0;
    // strategy moze byt Linear, Logarithmic alebo Exponential
    return strategy.calculate(getBaseProductionRate(), count)
        * productionMultiplier;
}
```

Keďže `ProductionStrategy` je `@FunctionalInterface`, stratégiu možno vyjadriť aj lambda výrazom — čo je moderná forma polymorfizmu v Java 8+ [6]. Stratégiu možno tiež zameniť za behu metódou `setStrategy(ProductionStrategy s)`, čo demonštruje flexibilitu polymorfizmu voči dedičnosti.

9.7 Rozhranie `GameObserver` — vzor `Observer`

Súbor: `src/main/java/com/idlefishing/observer/GameObserver.java`

Rozhranie `GameObserver` umožňuje registrovaným objektom (typicky UI panelom) reagovať na herné udalosti bez priamej závislosti na modeli. Je tiež deklarované ako `@FunctionalInterface`:

Listing 13: Rozhranie `GameObserver`

```
@FunctionalInterface
public interface GameObserver {
    void onEvent(GameEvent event);
}
```

`EventBus` udržiava mapu `Map<EventType, List<GameObserver>>`. Keď model zmení stav, zavolá:

Listing 14: Polymorfická notifikácia observerov v EventBus

```

public void publish(GameEvent event) {
    List<GameObserver> obs = listeners.getDefault(event.getType(), List.of()
);
    for (GameObserver o : obs) {
        o.onEvent(event); // runtime dispatch na konkrétneho observera
    }
}

```

Každý UI panel (`FishingPanel`, `ShopPanel`, `StatsPanel`, `LocationBar`) implementuje `GameObserver` vlastným spôsobom — jeden prekreslí scénu, druhý aktualizuje label, tretí prepočíta tlačidlá. Model pritom neimportuje žiadnu Swing triedu.

9.8 Zhrnutie polymorfizmu v projekte

Hierarchia	Typ	Počet impl.	Kľúčová kolekcia
Fish	abstraktná trieda	4	List<Fish> v FishingSpot
Vessel	abstraktná trieda	4	List<Vessel> v GameState
FishingSpot	abstraktná trieda	4	List<FishingSpot> v GameState
Upgrade	abstraktná trieda	2	List<Upgrade> v GameState
Command	rozhranie	5	Deque<Command> v CommandHistory
ProductionStrategy	rozhranie	3	pole strategy v Vessel
GameObserver	rozhranie	N	List<GameObserver> v EventBus
Spolu		22+	

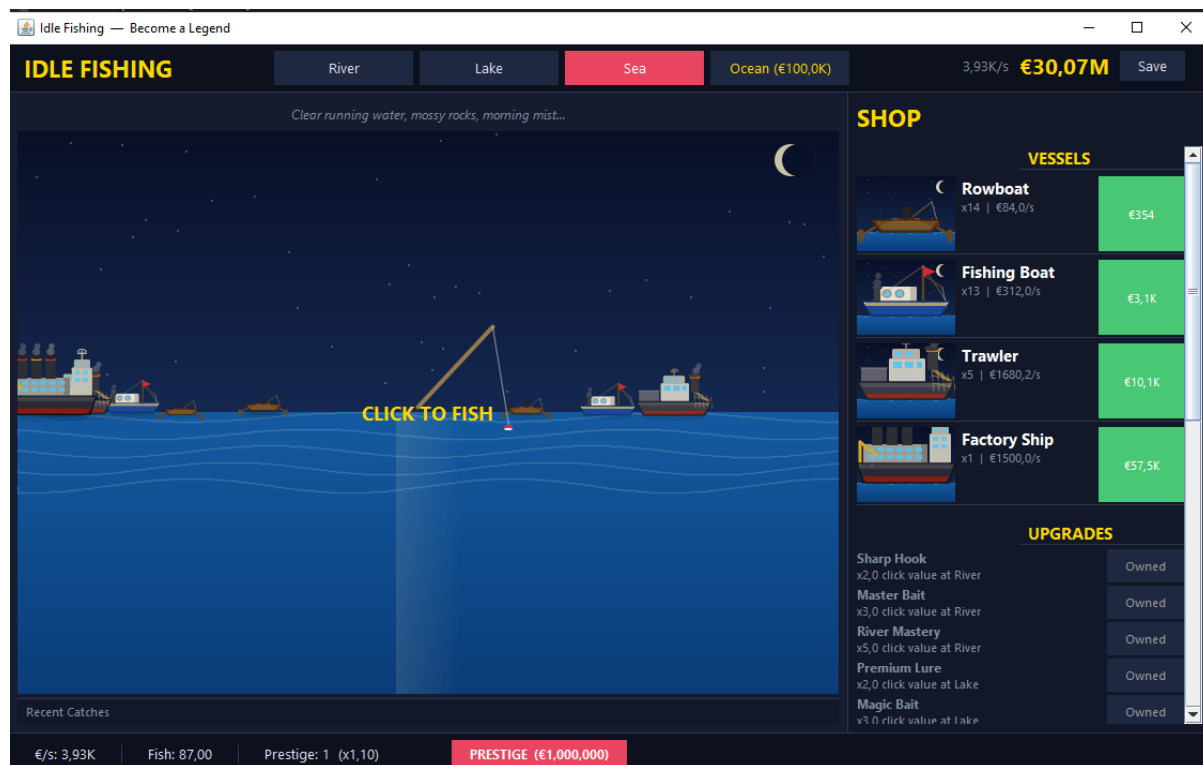
Tabuľka 9: Prehľad polymorfických hierarchií v projekte Idle Fishing

Polymorfizmus v projekte Idle Fishing výrazne znižuje väzbu (*coupling*) medzi komponentmi a zvyšuje rozšíriteľnosť systému. Nový druh ryby, nové plavidlo, nová lokalita alebo nová herná akcia môžu byť pridané bez úpravy existujúceho kódu — stačí vytvoriť novú podtriedu alebo implementáciu rozhrania. Tento princíp je v súlade s *Open/Closed Principle*, jednou zo SOLID zásad objektovo orientovaného návrhu [5].

10 Rozloženie UI

+-----+		
	LocationBar (4 tlačidla: River Lake Sea Ocean)	
+-----+		
	FishingPanel	ShopPanel (340 px)
	(animovaná scena)	- Plavidla (ikona + tlačidlo)
	- pozadie obloha	- Vylepsenia (scroll list)
	- voda + vlny	
	- lode sa kolisaju	
	- prud (klik = ryba)	
+-----+		
	StatsPanel (EUR/s ryby prestiz tlačidlo)	
+-----+		

10.1 GUI screenshot



Pozície lodí v scéne

Lode sú rozmiestené na predefinovaných pozíciách (ako zlomok šírky panela). Stredová oblasť (45–58 % šírky) je voľná pre prút. Lode sa animujú sínusovým kolísaním s periódou približne 2,4 s.

Plavidlo	x (zlomok šírky)	Škála
Rowboat ($\times 3$)	0,20 / 0,30 / 0,62	0,52
FishingBoat ($\times 2$)	0,14 / 0,72	0,64
Trawler ($\times 2$)	0,07 / 0,80	0,78
FactoryShip ($\times 2$)	0,04 / 0,85	0,95

Tabuľka 10: Pozície lodí v hernej scéne

11 Záver

Projekt Idle Fishing je plne funkčná idle-clicker hra napísaná v jazyku Java s grafickým rozhraním postaveným na knižnici Swing. Cieľom projektu bolo nielen vytvoriť zábavnú hru, ale predovšetkým dôsledne uplatniť princípy objektovo orientovaného programovania a osvedčené návrhové vzory.

Naplnenie cieľov

Projekt splnil všetky vytýčené ciele:

- **Plná hrateľnosť** — hráč môže rybařit, nakupovať plavidlá a vylepšenia, odomykať nové lokality a dosiahnuť prestíž.
- **Persistencia** — stav hry sa automaticky ukladá každých 60 sekúnd a načítava pri ďalšom spustení vrátane offline zárobkov.
- **Objektovo orientovaný návrh** — projekt má 7 polymorfických hierarchií s 22 konkrétnymi implementáciami, čo demonštruje hlboké uplatnenie dedičnosti, zapuzdrenia a polymorfizmu.
- **Návrhové vzory** — projekt implementuje 5 klasických návrhových vzorov: Singleton, Factory Method, Strategy, Template Method, Observer a Command.
- **Čistá architektúra** — vrstvená MVC-like štruktúra zabezpečuje, že model nie je závislý na UI a UI nie je závislé priamo na modeli — komunikácia prebieha výhradne cez `EventBus`.

Kľúčové technické rozhodnutia

Najvýznamnejším architektonickým rozhodnutím bolo použitie `EventBus` (vzor Observer) ako komunikačnej vrstvy medzi modelom a UI. Táto voľba umožňuje pridávať nové UI komponenty bez zásahu do modelu a testovať model izolovane.

Druhým kľúčovým rozhodnutím bolo oddelenie produkčného algoritmu do rozhrania `ProductionStrategy`. Rôzne matematické krivky rastu (lineárna, logaritmická, exponenciálna) vytvárajú rôznu hernú dynamiku pre každý typ plavidla, pričom trieda `Vessel` zostáva nezmenená.

Možné rozšírenia

V budúcich verziách by bolo možné:

- Pridať nové typy rýb alebo lokalít — stačí vytvoriť podtriedu `Fish` alebo `FishingSpot` bez zmeny existujúceho kódu.
- Implementovať funkciu *undo* — `CommandHistory` už uchováva históriu príkazov, chýba len rozšírenie rozhrania `Command` o metódu `undo()`.
- Pridať sieťovú tabuľku skóre — `SaveManager` by mohol byť rozšírený o vzdialené ukladanie bez zmeny herného modelu.
- Pridať zvukové efekty — nový `SoundObserver` implementujúci `GameObserver` by reagoval na udalosti bez akejkoľvek zmeny modelu.

Záverečné zhodnotenie

Idle Fishing demonštruje, že aj jednoduchá hra môže byť navrhnutá s dôrazom na kvalitu kódu, rozšíriteľnosť a dodržiavanie OOP princípov. Polymorfizmus je pritom ústredným mechanizmom, ktorý celú architektúru drží pohromade: umožňuje jednotný prístup k rôznorodým objektom, eliminuje opakovaný kód a robí systém pripravený na budúce rozšírenia bez porušenia existujúcej funkcionality.

Literatúra

- [1] Eckel, B. (2006). *Thinking in Java* (4th ed.). Prentice Hall. ISBN 978-0-13-187248-6.
- [2] Horstmann, C. S. (2018). *Core Java Volume I — Fundamentals* (11th ed.). Prentice Hall. ISBN 978-0-13-516630-7.
- [3] Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley. ISBN 978-0-13-468599-1.
- [4] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. ISBN 978-0-20-163361-5.
- [5] Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall. ISBN 978-0-13-449416-6.
- [6] Warburton, R. (2014). *Java 8 Lambdas: Pragmatic Functional Programming*. O'Reilly Media. ISBN 978-1-44-936972-7.
- [7] Oracle Corporation. (2023). *The JavaTM Tutorials — Polymorphism*. Dostupné online: <https://docs.oracle.com/javase/tutorial/java/IandI/polymorphism.html>
- [8] 3ntu Edu Java 2D graphics tutorial *Custom drawing code snippet*. Dostupné online: https://www3.ntu.edu.sg/home/ehchua/programming/java/J8b_Game_2DGraphics.html
- [9] Bro Code video *Java 2D canvas Graphics video*. Dostupné online: <https://www.youtube.com/watch?v=KcEvHq8Pqs0>